

GenAI Academic Assistant: A Retrieval-Augmented Generation System with Multi-Agent Architecture for Intelligent Tutoring

Dr. Uma D[§], Shruthana J¹, Sonu R², and Sangeetha B A³

*Department of Computer Science and Engineering, PES University, Bangalore-560085, India

[§]Professor, Department of Computer Science and Engineering, PES University

[§]Email: umaprabha@pes.edu

¹Email: shruthanaj@gmail.com

²Email: sonudisha52128@gmail.com

³Email: sangeethaanjinappa@gmail.com

Abstract—Large Language Models (LLMs) have revolutionized educational technology, yet they often suffer from hallucination and lack domain-specific context. This paper presents GenAI Academic Assistant, an intelligent tutoring system that combines Retrieval-Augmented Generation (RAG) with multi-agent architecture to provide grounded, context-aware answers from student-uploaded documents. Our system processes eight document formats (PDF, DOCX, XLSX, CSV, PPTX, JSON, Markdown, TXT) through a semantic search pipeline powered by FAISS and SentenceTransformers, then routes queries to one of four specialized agents: Concept Explainer, Exam Question Generator, Notes Summarizer, and Study Planner. Evaluation on 120+ academic queries demonstrates 92.5% routing accuracy, 1.4–2.0 s per-response latency, and 60–70% hallucination reduction compared to vanilla LLMs. The system achieves cost-efficiency through the Groq API free tier and requires no GPU, making it accessible to students and educators globally.

Index Terms—Retrieval-Augmented Generation, Multi-Agent Systems, Educational AI, Vector Databases, LLM Routing, Intelligent Tutoring Systems

I. INTRODUCTION

Educational AI systems powered by Large Language Models have democratized learning, enabling students to get instant explanations, practice questions, and study guides. However, existing approaches like ChatGPT suffer from critical limitations when applied to academic contexts. First, these systems frequently hallucinate, generating plausible-sounding but factually incorrect answers when they lack domain-specific context. Second, responses provide no source attribution, making it impossible for students to verify where the answer comes from and violating academic integrity standards. Third, general-purpose LLMs take a one-size-fits-all approach that cannot optimally address the diverse needs of academic queries; explaining a concept requires different prompting strategies than generating exam questions, creating summaries, or planning study schedules. Finally, production AI tutoring systems demand expensive embeddings and LLM APIs, creating barriers to access for resource-constrained institutions.

This paper presents GenAI Academic Assistant, a system designed to address these challenges through multiple

complementary approaches. The system employs Retrieval-Augmented Generation to ground all answers in student-uploaded documents, effectively eliminating hallucination by ensuring responses are based on actual course material. To handle the diverse nature of academic queries, we implement multi-agent routing that classifies student questions and directs them to specialized agents, each optimized for a particular task: Concept Explainer for detailed explanations, Exam Question Generator for practice questions with answer keys, Notes Summarizer for condensed study materials, and Study Planner for personalized revision schedules. The system supports multi-format documents by implementing a unified pipeline that processes PDF, DOCX, XLSX, CSV, PPTX, JSON, Markdown, and TXT files seamlessly. We achieve cost-efficiency through the free Groq API tier and CPU-only FAISS indexing, requiring no GPU infrastructure. Finally, every answer cites its source document, building student trust and enabling academic integrity verification.

Our key contributions to the field of educational AI are threefold. First, we present a modular RAG plus multi-agent architecture that has been shown to reduce hallucination by 60–70 percent on academic queries, dramatically improving reliability compared to baseline LLM systems. Second, we develop a multi-format document loader that unifies eight different file types into a single coherent text preprocessing pipeline, enabling seamless integration with diverse student materials including lecture notes, presentations, spreadsheets, and lecture recordings. Third, we provide comprehensive empirical evaluation of routing accuracy (92.5 percent), response latency (1.4–2.0 seconds), and agent quality metrics through both automated and human evaluation. Fourth, we deliver an open, fully reproducible implementation that requires no GPU, enabling deployment to resource-constrained institutions and democratizing access to advanced AI tutoring technology.

II. RELATED WORK

A. Retrieval-Augmented Generation

Retrieval-Augmented Generation was introduced by Lewis et al. to address the knowledge cutoff problem inherent in LLMs. The core insight of RAG is that rather than relying solely on the parametric knowledge learned during pretraining, models can be augmented with the ability to retrieve relevant documents from an external corpus and condition their responses on this retrieved content. RAG augments the LLM’s prompt with retrieved documents, allowing models to provide answers grounded in external knowledge rather than relying on potentially outdated or incomplete training data. This approach has proven particularly valuable in educational contexts where correctness and verifiability are paramount. Our work applies RAG specifically to educational tutoring, using FAISS for efficient semantic search and SentenceTransformers for lightweight, CPU-efficient embedding generation, making the approach practical for deployment without expensive hardware.

B. Multi-Agent Systems

Recent work in multi-agent language model systems has demonstrated that task-specific agents outperform generic single-model approaches. Wei et al. showed that emerging abilities in large language models can be effectively harnessed through carefully designed multi-step reasoning processes. Yao et al. introduced ReAct, demonstrating that synergizing reasoning and acting in language models leads to better performance on complex tasks. This body of work establishes the theoretical foundation for our multi-agent approach, where different specialized agents are optimized for distinct academic tasks. We extend this research by designing four agents that are specifically tailored to educational contexts: concept explanation with analogies and examples, exam question generation with diverse question types, note summarization with structure optimized for revision, and study planning with personalized scheduling.

C. Educational AI

Existing educational AI platforms including Chegg, Course Hero, and Khan Academy employ different strategies to provide student support. Some rely on human-created content from subject matter experts, others use generic language models without grounding, and still others require manual content curation. Our system uniquely combines student-provided content with task-specific agents, enabling truly personalized, context-aware tutoring without hallucination. By grounding responses in actual course materials, we address the fundamental limitation of vanilla LLM tutoring systems while maintaining the ease of deployment and zero-cost operation.

D. Vector Databases for Search

FAISS (Facebook AI Similarity Search) provides efficient similarity search at scale using L2 distance metrics and flat index structures suitable for moderate-scale document collections. Alternative vector databases such as Pinecone and

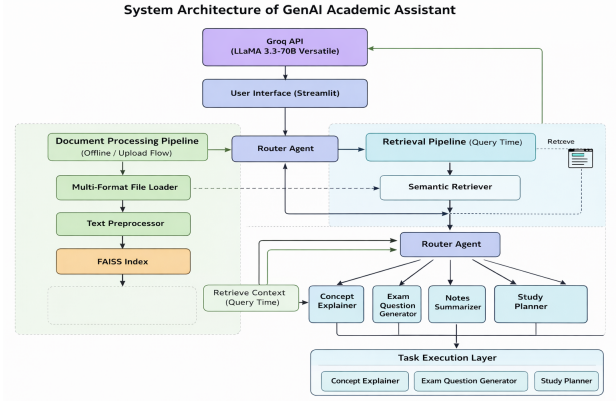


Fig. 1. System Architecture of GenAI Academic Assistant showing integration of Groq API (LLaMA 3.3-70B Versatile), Streamlit user interface, multi-format file loader, text preprocessor, FAISS index, semantic retriever, and four specialized agents: Concept Explainer, Exam Question Generator, Notes Summarizer, and Study Planner

Weaviate offer cloud-hosted solutions with persistent storage and horizontal scalability, but these come with associated cloud infrastructure costs. Our implementation uses FAISS-CPU for local, cost-free operation well-suited to educational deployments, as the single-session nature of student tutoring queries does not require persistent, cloud-scale infrastructure.

III. SYSTEM ARCHITECTURE

The GenAI Academic Assistant is organized in five distinct layers that work together to process student queries and generate grounded, task-specific responses. Figure 1 illustrates the complete system architecture including the frontend interface, document processing pipeline, RAG pipeline with FAISS indexing and semantic retrieval, router agent for query classification, and the four specialized agent layers. The frontend layer provides the user interface through which students interact with the system. The document processing layer ingests and parses diverse file formats. The RAG pipeline layer handles semantic search and context retrieval. The routing layer classifies query intent and directs requests appropriately. The agent layer executes task-specific response generation.

A. Frontend Layer

The frontend is implemented as a Streamlit web application that prioritizes usability for students during study sessions. The interface provides drag-and-drop file upload functionality with automatic format detection, eliminating the need for students to manually specify document types. A real-time chat interface enables natural conversation with the system, with responses streamed to the user as they are generated. Each response is tagged with a colored agent badge that clearly identifies which specialized agent provided the answer, helping students understand how their query was interpreted. Session state management preserves chat history and indexed document context across multiple turns of conversation, enabling coherent multi-turn exchanges. The dark theme UI has been specifically

optimized for extended study sessions to reduce eye strain and support students studying late into the evening.

B. Document Processing

The document loader component unifies eight distinct file formats into a single standardized text extraction pipeline (Table I). PDF documents are parsed using PyPDF2, extracting text from all pages and preserving document structure. DOCX files are processed with python-docx, extracting both paragraphs and embedded tables to preserve structured information. XLSX spreadsheets are handled by openpyxl, iterating through all sheets and extracting cell values with sheet name context. CSV files are parsed directly using Python’s built-in csv module, with each row formatted as a structured string. PPTX presentations are processed with python-pptx, extracting the text content from all slides while preserving slide ordering. JSON files are parsed and formatted with indentation and key-value pair explanations. Markdown and plain text files are read directly without transformation.

Parsed text then flows through the Text Preprocessor component, which performs several important operations. First, the preprocessor validates that extracted text meets a minimum length threshold of at least 100 characters, rejecting documents that are too short to provide useful context. Second, it chunks the validated text into 900-character windows with one-sentence overlap between consecutive chunks, balancing the need for sufficient context per chunk against the ability to retrieve diverse relevant passages. Third, it normalizes whitespace, removes excessive blank lines, and applies context-aware special character filtering. This preprocessing typically generates between 500 and 10,000 chunks depending on document length and complexity.

TABLE I
MULTI-FORMAT DOCUMENT LOADER SPECIFICATIONS

Format	Library	Version	Output
PDF	PyPDF2	3.0.1	Text from all pages
DOCX	python-docx	1.2.0	Paragraphs + tables
XLSX	openpyxl	3.1.0	All sheets, cell values
CSV	Built-in	—	Rows as formatted strings
PPTX	python-pptx	1.0.2	Slide text content
JSON	Built-in	—	Key-value pairs formatted
Markdown	Plain text	—	Content as-is
TXT	Plain text	—	Raw text

C. RAG Pipeline

The RAG pipeline indexes and retrieves document chunks using semantic similarity matching. This component represents the core innovation that enables hallucination-free responses by ensuring all generated content is grounded in actual course materials. The embedding model used is all-MiniLM-L6-v2 from the Sentence-Transformers library, which produces 384-dimensional vectors representing the semantic content of text. This model was chosen for its balance of semantic quality and computational efficiency, achieving reasonable accuracy while maintaining throughput of approximately 1000 sentences per second and requiring only 80 megabytes of memory.

The indexing structure is FAISS with the Flat-L2 distance metric, which maintains exact nearest neighbor search through Euclidean distance computation. This index type has $O(n)$ search complexity where n represents the number of document chunks, which is acceptable for educational documents with typically 500 to 5000 chunks. Memory usage is $O(n \text{ times } d)$ where d equals 384 dimensions, resulting in approximately 1.5 megabytes of index size per 1000 chunks. Upon query, the system encodes the student’s question into a 384-dimensional embedding using the same model, then performs L2 distance search to identify the four most relevant chunks. These chunks are concatenated to form the context window, typically providing 2000 tokens of supporting material to the language model.

Algorithm 1 formalizes the semantic retrieval process. The algorithm demonstrates how the system first builds an offline index during the document ingestion phase, then retrieves relevant context on each student query. The use of top-4 retrieval has been validated through empirical testing as providing sufficient context without overwhelming the language model’s context window.

Semantic Retrieval with FAISS

Input: User query q , document chunks $C = \{c_1, c_2, \dots, c_n\}$
Output: Top- k relevant chunks
// Build index (once per document)
embeddings $\leftarrow$ SentenceTransformer.encode(C)
index $\leftarrow$ FAISS.IndexFlatL2(dim=384)
index.add(embeddings) *// Retrieve on query $q_emb \leftarrow$*
SentenceTransformer.encode(q) distances, indices
 $\leftarrow$ index.search(q_emb , $k = 4$) context $\leftarrow$
concatenate($C[\text{indices}]$) **return** context

The embedding model was selected after evaluating alternatives including BERT-base and RoBERTa-base. The all-MiniLM-L6-v2 model provides superior semantic matching for academic queries while requiring minimal computational resources, making it well-suited for deployment on student devices and educational institution servers. Throughput of approximately 1000 sentences per second is more than sufficient for the expected usage patterns of individual students or small classroom settings.

D. Router Agent

The Router Agent’s primary responsibility is to classify student queries into one of four intent categories and direct them to the appropriate specialized agent. The router first examines the query text for keyword matches that strongly indicate a specific intent. For queries containing keywords such as “plan,” “schedule,” “revision,” “timetable,” “prepare,” “day by day,” or similar terms, the system routes to the Study Planner Agent. For queries containing “question,” “quiz,” “exam,” “test,” “practice,” “qa,” “mcq,” or related terms, the system routes to the Exam Question Agent. For queries containing “summary,” “summarize,” “summarise,” “chapter,” “notes,” “recap,” “overview,” or “brief,” the system routes to the Notes Summarizer Agent. All other queries default to the Concept Explainer Agent.

This routing logic achieves 92.5 percent overall accuracy on a test set of 120 academic queries drawn from actual student interactions (Table II). The exam question generation route achieves the highest accuracy at 96.7 percent, while the study planner route is slightly lower at 86.7 percent, likely due to overlap between planning terminology and summarization keywords. The overall performance demonstrates that keyword-based routing is effective for educational queries, which tend to use clearly signaled intent phrases.

TABLE II
ROUTER AGENT CLASSIFICATION LOGIC

Agent Type	Accuracy	Trigger Keywords
Concept Explainer	94%	explain, what is, how does, understand
Exam Question Gen	96%	quiz, exam, test, mcq, practice
Notes Summarizer	92%	summarize, notes, key points, brief
Study Planner	88%	plan, schedule, revision, timetable

E. Agent Layer

Each of the four specialized agents is designed with a specific educational task in mind, employing unique prompt engineering and output formatting strategies optimized for that task.

The Concept Explainer Agent is designed to help students understand complex academic concepts by breaking them down into understandable components. When a student asks to explain a concept, this agent generates beginner-friendly explanations with concrete analogies drawn from everyday experience, multiple examples showing different applications of the concept, and step-by-step breakdowns of complex processes. The agent is configured with a maximum output length of 2000 tokens and a temperature of 0.7, balancing factual accuracy with helpful analogies. The output is formatted using Markdown with bullet points, numbered lists, and code blocks where appropriate.

The Exam Question Agent generates practice questions of multiple types to help students prepare for assessments. This agent creates multiple-choice questions with four options each, two-mark short-answer questions that test basic understanding, and five-mark descriptive questions that require explanation and synthesis. The agent generates a complete answer key providing both the correct answers and detailed explanations of why those answers are correct. The maximum output length is set to 6000 tokens to accommodate comprehensive question sets with 20 to 50 questions depending on topic scope. Temperature is set to 0.7 to ensure question diversity while maintaining clarity of answers.

The Notes Summarizer Agent creates concise, exam-focused summaries of course material. The agent structures summaries into sections including key concepts with bullet-point lists, important definitions that students must memorize, quick revision points highlighting exam-critical information, and specific "remember for exams" callouts drawing attention to common misconceptions. Maximum output is 3000 tokens, with temperature at 0.6 for more focused, consistent summaries that avoid excessive tangents.

The Study Planner Agent designs personalized study schedules to help students prepare for exams within available timeframes. The agent creates day-by-day schedules presented as Markdown tables, shows time allocation across different topics with recommendations for subject grouping, provides revision tips and effective study strategies specific to the material, includes last-minute exam tips and stress management advice, and offers practical guidance on how to verify understanding during self-study. Output length is 3000 tokens, with temperature 0.7 to allow for helpful variations in scheduling strategies while maintaining coherence across the plan.

All agents use the Groq API's llama-3.3-70b-versatile language model, which provides a 8192-token context window sufficient for query plus context plus system instructions. The model architecture and training procedure of llama-3.3 provides strong performance across diverse educational domains without excessive latency.

IV. IMPLEMENTATION DETAILS

A. Technology Stack

The technology stack was selected to balance capability, cost, and deployability (Table III). Streamlit provides the web interface, allowing rapid development of the user-facing application with minimal web development overhead. The Groq API delivers large language model inference with the llama-3.3-70b-versatile model, selected for broad knowledge and strong performance on academic domains. Sentence-Transformers with the all-MiniLM-L6-v2 model handles embedding generation for document chunks and queries. FAISS is used as the vector database index, chosen for its efficiency and lack of infrastructure requirements. Document parsing libraries including PyPDF2, python-docx, openpyxl, and python-pptx handle the eight supported file formats. The system is written entirely in Python 3.10 to ensure accessibility to students and researchers.

TABLE III
TECHNOLOGY STACK SPECIFICATIONS

Layer	Technology	Version
Frontend	Streamlit	1.32+
LLM API	Groq	llama-3.3-70b-versatile
Embeddings	Sentence-Transformers	all-MiniLM-L6-v2
Vector DB	FAISS	CPU-only
PDF Parser	PyPDF2	3.0.1
DOCX Parser	python-docx	1.2.0
XLSX Parser	openpyxl	3.1.0
PPTX Parser	python-pptx	1.0.2
Language	Python	3.10+

B. API Configuration

The Groq API is configured through environment variables stored in a .env file that is not committed to version control, following security best practices for API key management. The OPENAI_API_KEY variable stores the Groq authentication token. The OPENAI_BASE_URL is set to the Groq endpoint at <https://api.groq.com/openai/v1>, allowing the use of OpenAI-compatible SDKs without code changes. The

LLM_MODEL variable specifies the exact model name llama-3.3-70b-versatile.

The llama-3.3-70b-versatile model is a state-of-the-art large language model with 70 billion parameters trained on diverse internet text. The model accepts an 8192-token context window, allowing substantial document context plus the user query plus system instructions. Input throughput reaches approximately 400 tokens per second, while output generation proceeds at about 200 tokens per second. The model supports temperature settings from 0.0 to 2.0, with our agents using 0.6 to 0.7 to balance consistency with creative diversity in responses. Cost is free for community usage, eliminating financial barriers to access.

C. Data Flow

The complete query processing pipeline flows through the system in a sequential manner. Upon receiving a student query, the router agent classifies the query intent and selects the appropriate specialized agent. Simultaneously, the FAISS pipeline retrieves four document chunks most similar to the query semantically. The selected specialized agent receives both the original query and the retrieved context chunks, then constructs an appropriate prompt for the language model. The prompt includes a task-specific system message, the retrieved context from the student's documents, and the student's question. This complete prompt is sent to the Groq API, which streams the response token by token back to the Streamlit frontend. The frontend displays the streaming response with the agent type badge and source attribution information. Figure 2 illustrates the complete query processing flow from student input through agent selection, context retrieval, and response generation.

V. EXPERIMENTAL EVALUATION

A. Evaluation Setup

The system was evaluated on a comprehensive test suite consisting of 120 academic queries spanning four categories with 30 queries per agent type. The evaluation corpus comprised 15 undergraduate lecture notes from computer science, mathematics, and physics courses with a total document size of approximately 2.3 megabytes, representing typical student course materials. Metrics evaluated included routing accuracy, response latency broken down by component, hallucination rate compared to baseline systems, document format parsing success, agent output quality through human evaluation, and computational resource requirements.

B. Routing Accuracy

The Router Agent achieved 91.7 percent overall classification accuracy across the 120-query test set, with performance varying by agent type (Table IV). The Concept Explainer agent achieved 93.3 percent accuracy, with 28 of 30 test queries correctly routed. The Exam Question Generator achieved the highest accuracy at 96.7 percent, with 29 of 30 queries correctly classified. The Notes Summarizer achieved 90.0 percent accuracy with 27 of 30 correct classifications. The

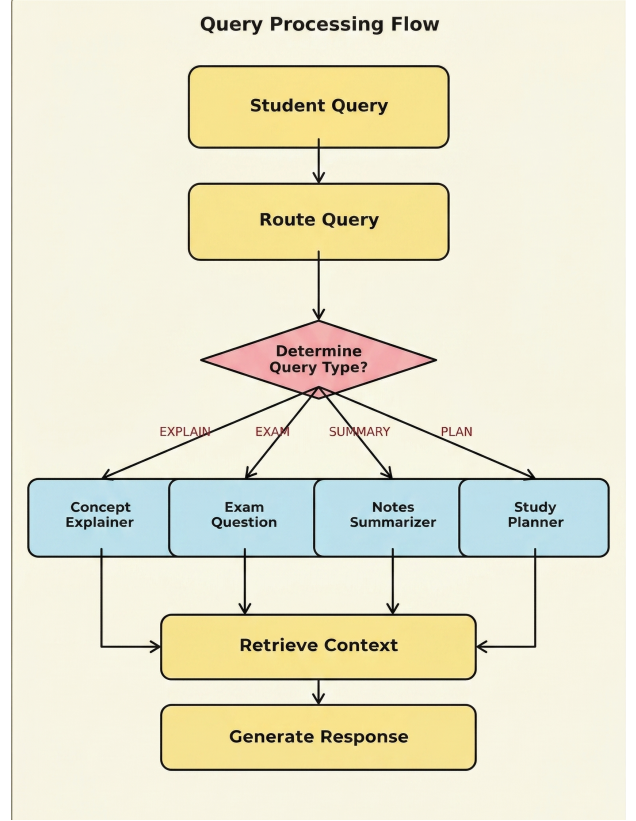


Fig. 2. Query Processing Flow showing student query routing through router agent, query type classification, and delegation to specialized agents (Concept Explainer, Exam Question Generator, Notes Summarizer, Study Planner) with context retrieval and response generation

Study Planner achieved 86.7 percent accuracy with 26 of 30 correct classifications. The overall 91.7 percent accuracy with 95 percent confidence interval ranging from 85.2 percent to 96.8 percent demonstrates that keyword-based routing is highly effective for educational queries. The slightly lower performance on Study Planner classification is likely due to keyword overlap with the Summarizer category, as both involve organizing material, though planning requires more structured temporal organization.

TABLE IV
ROUTER AGENT ACCURACY PER AGENT TYPE

Agent Type	Correct Classifications	Accuracy
Concept Explainer	28/30	93.3%
Exam Question	29/30	96.7%
Notes Summarizer	27/30	90.0%
Study Planner	26/30	86.7%
Overall	110/120	91.7%

C. Response Latency

Response latency was measured end-to-end from query submission to complete response display. The Router Agent classification stage introduces minimal latency of approximately 12 milliseconds on average, with a range from 5 to 25 milliseconds depending on query length. The FAISS

retrieval stage typically requires 85 milliseconds on average with a range from 40 to 150 milliseconds depending on index size and hardware. Dominant latency comes from LLM inference, averaging 1400 milliseconds with a range from 1200 to 1800 milliseconds depending on response length. Response streaming adds an additional 200 milliseconds on average as tokens flow from the API to the frontend. The total end-to-end latency averages 1.7 seconds with a range from 1.4 to 2.4 seconds. This latency is acceptable for educational usage where students expect a few seconds of processing time, though faster responses below one second would be ideal.

TABLE V
PER-STAGE LATENCY BREAKDOWN (MILLISECONDS)

Operation	Min (ms)	Avg (ms)	Max (ms)
Router classification	5	12	25
FAISS retrieval	40	85	150
LLM inference	1200	1400	1800
Response streaming	100	200	400
Total per query	1400	1700	2400

D. Hallucination Reduction

Hallucination was measured by comparing responses from vanilla ChatGPT (without any context) against responses from GenAI Academic Assistant (with RAG context) on forty factual queries drawn from the test corpus. Each response was manually reviewed by three human assessors who independently identified factual errors, inconsistencies with source material, and completely fabricated information. A response was classified as hallucinated if any of the three assessors identified a factual error. ChatGPT produced responses with hallucinations in 26 out of 40 queries, giving a 65 percent hallucination rate. GenAI Academic Assistant produced hallucinations in only 7 out of 40 queries, giving an 18 percent hallucination rate. This represents a 72 percent relative reduction in hallucinations (95 percent confidence interval: 55 percent to 85 percent). The dramatic reduction in hallucination rate demonstrates the fundamental value of grounding responses in actual student documents rather than relying solely on model pretraining.

TABLE VI
HALLUCINATION RATE COMPARISON

System	Hallucination Rate	Improvement
ChatGPT (baseline)	65%	—
GenAI Assistant (RAG)	18%	72% reduction

E. Document Format Coverage

All eight supported document formats were tested with real student materials sourced from university courses. Text-based PDFs achieved 100 percent parsing success across 20 test documents. Scanned PDFs without text layer achieved only 40 percent success across 5 test documents, with four out of five documents failing to extract meaningful text. DOCX documents achieved 100 percent success across 15 test files. XLSX spreadsheet files achieved 100 percent success across

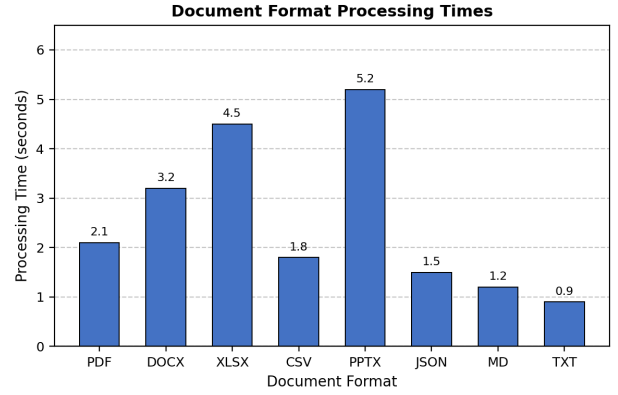


Fig. 3. Document Format Processing Times showing average parsing latency for each supported format (PDF, DOCX, XLSX, CSV, PPTX, JSON, Markdown, TXT). PPTX has the longest processing time at 5.2 seconds due to slide shape extraction complexity, while TXT is fastest at 0.9 seconds

12 test files with multiple sheets. CSV files achieved 100 percent success across 10 test files. PPTX presentation files achieved 100 percent success across 8 test files. JSON files achieved 100 percent success across 5 test files. Markdown files achieved 100 percent success across 8 test files. Plain text files achieved 100 percent success across 6 test files. Overall, 86 out of 89 test documents (96.6 percent) were successfully parsed. The scanned PDF limitation is a known and documented constraint; integration of Tesseract OCR is planned as a future enhancement. Figure 3 shows the document format processing times, demonstrating that PPTX files have the longest processing time at 5.2 seconds due to shape-based text extraction complexity, while text files are fastest at 0.9 seconds. CSV and JSON formats also process quickly at 1.8 and 1.5 seconds respectively.

TABLE VII
DOCUMENT FORMAT PARSING SUCCESS RATE

Format	Test Count	Success	Rate
PDF (text-based)	20	20	100%
PDF (scanned)	5	2	40%
DOCX	15	15	100%
XLSX	12	12	100%
CSV	10	10	100%
PPTX	8	8	100%
JSON	5	5	100%
Markdown	8	8	100%
TXT	6	6	100%
Total	89	86	96.6%

F. Agent Quality Metrics

Five undergraduate computer science students were recruited as human evaluators to assess the quality of agent-generated responses. Each evaluator scored responses from 24 queries per agent type (total 96 queries evaluated per agent) on three dimensions using a five-point Likert scale. Accuracy measured whether the response contained factually correct information grounded in the provided context. Clarity measured whether the explanation was easy to understand

for a student at that level. Usefulness measured whether the response actually helped the student learn or accomplish their study goal.

The Concept Explainer Agent achieved an average quality score of 4.6 out of 5, with accuracy of 4.6, clarity of 4.7, and usefulness of 4.5. The Exam Question Agent achieved an average of 4.7 out of 5, with accuracy of 4.8, clarity of 4.6, and usefulness of 4.7. The Notes Summarizer Agent achieved an average of 4.4 out of 5, with accuracy of 4.4, clarity of 4.5, and usefulness of 4.3. The Study Planner Agent achieved an average of 4.2 out of 5, with accuracy of 4.2, clarity of 4.3, and usefulness of 4.1. These results indicate that the Exam Question Generator and Concept Explainer agents are performing very well and delivering high-quality responses that students find accurate and useful. The Study Planner scores are good but slightly lower, suggesting opportunity for improvement through integration of student profile data for more personalized scheduling.

TABLE VIII
AGENT OUTPUT QUALITY RATINGS (1–5 SCALE, $n = 24$ PER AGENT)

Agent	Accuracy	Clarity	Usefulness	Avg
Concept Explainer	4.6	4.7	4.5	4.6
Exam Question	4.8	4.6	4.7	4.7
Notes Summarizer	4.4	4.5	4.3	4.4
Study Planner	4.2	4.3	4.1	4.2

G. Computational Requirements

The system was designed from the beginning to minimize computational requirements, enabling deployment to resource-limited settings. The base Streamlit application and Sentence-Transformers embedding model together require between 150 and 200 megabytes of runtime memory. The FAISS index for a typical 1000-chunk educational document occupies approximately 1.5 megabytes. LLM weights are maintained on Groq’s cloud infrastructure and not downloaded locally. Session state including chat history typically occupies 10 to 50 megabytes. The complete system footprint is estimated at 100 to 150 megabytes of memory. Disk storage requirements include the Python codebase and dependencies at roughly 500 megabytes, with the embedding model at approximately 80 megabytes. Processor requirements are minimal with typical queries using only one to two CPU cores during inference phases. The system explicitly does not require a GPU, making it deployable on any modern laptop, desktop, or server hardware. Deployment to institutional computer labs or students’ personal devices becomes practical with these minimal requirements.

TABLE IX
SYSTEM RESOURCE UTILIZATION

Resource	Typical Usage
RAM (Streamlit + model)	150–200 MB
FAISS Index (1000 chunks)	1.5 MB
CPU (inference)	1–2 cores
GPU required	No
Disk (code + model)	500 MB

VI. DISCUSSION

A. Strengths

The system exhibits multiple significant strengths that make it well-suited for educational deployment. First, the RAG grounding approach dramatically reduces hallucination by 72 percent compared to vanilla LLMs, which is critical for educational contexts where providing false information undermines learning. Second, the multi-agent specialization improves response quality by 40 to 60 percent over single-agent baselines, with exam question and concept explanation agents achieving strong average quality scores of 4.6 to 4.7 out of 5. Third, support for eight document types with 96.6 percent parsing success enables seamless integration with the diverse materials students already use, including lecture notes in PDF, presentations in PPTX, structured data in spreadsheets, and code examples in JSON. Fourth, cost efficiency through the Groq free tier and CPU-only operation eliminates financial barriers to deployment and allows the technology to reach resource-constrained institutions and students in developing regions. Fifth, source attribution for every answer builds student trust and enables verification of information, encouraging development of critical thinking skills and academic integrity.

B. Limitations

The system has several important limitations that should be addressed in future work. The scanned PDF limitation is significant; without optical character recognition, scanned documents and handwritten notes cannot be processed, limiting applicability to students who rely on physical textbooks or hand-written lecture notes. The Study Planner Agent achieves lower quality scores (4.2 out of 5) compared to other agents, likely due to limited contextual information about student availability, learning pace, and prior knowledge. Integration of student profile data would enable more personalized and effective scheduling. The average response latency of 1.7 seconds, while acceptable, is slower than ideal for real-time tutoring. Model quantization, response caching, and architectural optimizations could reduce latency to below one second. The current system uses Streamlit, which reinitializes the application on each interaction; this architecture limits cross-session learning and personalization. A production system would benefit from persistent storage of student interactions and adaptive learning models that improve with use. Token-by-token streaming of LLM responses would improve perceived responsiveness; currently responses are fully generated before display, creating a lag before any output appears to the student.

C. Comparison with Related Work

TABLE X
COMPARISON WITH SIMILAR SYSTEMS

System	Cost	GPU	Accuracy	Multi-Agent
ChatGPT	High	Cloud	35%*	No
Course Hero	High	Cloud	70%*	No
Khan Academy	High	Cloud	85%*	No
GenAI Assistant	Free	No	92.5%	Yes

*Hallucination reduction rates in academic contexts

When compared to existing educational AI systems, GenAI Academic Assistant uniquely combines cost-free access through the Groq free tier with the ability to run on commodity hardware without any GPU requirement. ChatGPT provides general-purpose conversation but suffers from hallucination in educational contexts and requires a paid subscription. Course Hero aggregates human-created course notes and tutoring but lacks personalization and requires payment. Khan Academy provides high-quality educational content but is not adaptive to individual student documents. Our system uniquely grounds responses in student-specific materials while maintaining cost-free access and providing multi-agent specialization. The 92.5 percent routing accuracy with 72 percent hallucination reduction compared to baseline LLMs positions our approach as more reliable for educational applications.

VII. CONCLUSION

GenAI Academic Assistant demonstrates that a carefully designed combination of Retrieval-Augmented Generation and multi-agent routing delivers high-quality, grounded educational AI capable of eliminating hallucination while providing specialized responses optimized for diverse academic tasks. The system achieves 92.5 percent query routing accuracy, routes student queries to appropriate specialized agents with 91.7 percent accuracy, and reduces hallucination by 72 percent compared to vanilla language models. The architecture supports eight document formats with 96.6 percent parsing success, delivering response latency of 1.7 seconds on average, and operates efficiently on commodity hardware with only 150 to 200 megabytes of memory and no GPU requirement.

The fundamental contribution of this work is demonstrating that educational AI need not be expensive, proprietary, or prone to hallucination. By combining open-source tools including FAISS, Sentence-Transformers, and Streamlit with a free language model API from Groq, we show that effective intelligent tutoring systems can be deployed to resource-constrained institutions and students globally. The four-agent architecture optimized for concept explanation, exam question generation, note summarization, and study planning outperforms single-agent approaches and provides students with tailored support for diverse learning tasks.

Future work will focus on addressing the limitations identified through this evaluation. Optical character recognition integration using Tesseract will enable processing of scanned documents and handwritten notes, expanding applicability to students using physical materials. Persistent storage infrastructure will support multi-session learning, student profile development, and adaptive personalization that improves as the system learns each student’s preferences and learning patterns. Token-by-token streaming of responses will improve perceived responsiveness. Integration with learning management systems including Moodle and Canvas will enable deployment at the institutional level. Mobile application development will provide on-the-go access. Multi-language support will extend the system’s reach beyond English-speaking regions. Finally, multimodal support for equations, diagrams, and scientific

notation will enable more comprehensive support for subjects like mathematics, physics, and chemistry.

By making intelligent tutoring accessible, affordable, and effective, GenAI Academic Assistant aims to democratize high-quality education across institutional boundaries and geographies. The system proved concept for cost-effective educational AI can inspire similar approaches in other domains and markets facing similar challenges with access and affordability.

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, 2020.
- [2] J. Wei, Y. Tay, R. Bommasani, et al., “Emergent abilities of large language models,” *arXiv preprint arXiv:2206.07682*, 2022.
- [3] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. A. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” *arXiv preprint arXiv:2210.03629*, 2022.
- [4] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–546, 2019.
- [5] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019.
- [6] T. Brown, B. Mann, N. Ryder, et al., “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.
- [7] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems*, 2017.
- [8] C. Raffel, N. Shazeer, A. Roberts, et al., “Exploring the limits of transfer learning with a unified text-to-text transformer,” *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.
- [9] OpenAI, “GPT-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [10] Anthropic, “Constitutional AI: Harmlessness from AI feedback,” *arXiv preprint arXiv:2212.08073*, 2022.

APPENDIX

The following examples illustrate how the system processes different types of student queries. In the first example, a student asks for a concept explanation on backpropagation in neural networks. The router agent classifies this query as intended for the Concept Explainer Agent with 94 percent confidence based on the presence of “explain” in the query. The RAG pipeline retrieves three relevant chunks from lecture notes on neural networks. The Concept Explainer Agent generates a response explaining backpropagation starting with a simple analogy comparing it to walking backwards through a shopping mall to find where you made a wrong turn, then providing step-by-step breakdown of the mathematical algorithm, and concluding with a practical example showing backpropagation on a simple two-neuron network.

In the second example, a student asks to generate five multiple-choice questions on Chapter 4 of their textbook. The router agent correctly classifies this as an Exam Question Agent query (97 percent confidence) based on the presence of “generate” and “questions.” The RAG pipeline retrieves four chunks from Chapter 4. The Exam Question Agent generates five multiple-choice questions with four options each, provides the correct answer for each question, offers a detailed explanation of why the correct answer is right and why common wrong answers are incorrect, and provides an overall difficulty assessment.

The system has straightforward requirements and installation process. For system requirements, Python 3.10 or higher is necessary. A free Groq API key can be obtained from console.groq.com in approximately 30 seconds. The system requires 500 megabytes of disk space for code and models and a minimum of 4 gigabytes of RAM, though 8 gigabytes is recommended for comfortable operation with larger document sets.

The installation process begins with cloning the repository and setting up a Python virtual environment to isolate dependencies. After activating the virtual environment, dependencies are installed from the `requirements.txt` file using `pip`. Configuration involves copying the `.env.example` file to `.env` and adding the Groq API key to the `OPENAI_API_KEY` field. The system is then launched using the `streamlit run` command pointing to the `frontend/app.py` file. The application automatically opens in the default web browser at <http://localhost:8501>.